

NYC Crash Data Cleaning

Aansh Jha

```
from pathlib import Path
import json

import numpy as np
import pandas as pd
from matplotlib.path import Path as PolygonPath

DATA_DIR = Path("data")
CRASH_FILE = DATA_DIR / "nyc_crashes_2024-06-30_to_2024-07-06.csv"
```

1. Load the crash data

The file contains the seven-day period from June 30 through July 6, 2024. Dates are parsed at load time, crash times are retained as strings, and ZIP codes are loaded as strings so leading zeroes cannot be lost.

```
df = pd.read_csv(
    CRASH_FILE,
    parse_dates=["crash_date"],
    dtype={"crash_time": "string", "zip_code": "string"},
)

print(f"Rows: {len(df):,}")
print(f>Date range: {df['crash_date'].min().date()} to {df['crash_date'].max().date()}")
df[["crash_date", "crash_time", "borough", "zip_code", "latitude", "longitude"]].head()
```

Rows: 1,676

Date range: 2024-06-30 to 2024-07-06

	crash_date	crash_time	borough	zip_code	latitude	longitude
0	2024-06-30	0:00	NaN	<NA>	40.638268	-74.07880
1	2024-06-30	0:00	BRONX	10455	40.815754	-73.89529
2	2024-06-30	0:00	MANHATTAN	10004	40.704834	-74.01658
3	2024-06-30	0:04	NaN	<NA>	40.712490	-73.96854
4	2024-06-30	0:05	NaN	<NA>	40.793980	-73.97229

2. Clean the variable names

```
df.columns = (  
    df.columns.str.strip()  
    .str.lower()  
    .str.replace(r"[^a-z0-9]+", "_", regex=True)  
    .str.replace(r"(?<=code)(\d)$", r"_1", regex=True)  
    .str.strip("_")  
)  
  
print("\n".join(df.columns))
```

```
crash_date  
crash_time  
borough  
zip_code  
latitude  
longitude  
location  
on_street_name  
off_street_name  
cross_street_name  
number_of_persons_injured  
number_of_persons_killed  
number_of_pedestrians_injured  
number_of_pedestrians_killed  
number_of_cyclist_injured  
number_of_cyclist_killed  
number_of_motorist_injured  
number_of_motorist_killed  
contributing_factor_vehicle_1  
contributing_factor_vehicle_2  
contributing_factor_vehicle_3  
contributing_factor_vehicle_4  
contributing_factor_vehicle_5  
collision_id  
vehicle_type_code_1  
vehicle_type_code_2  
vehicle_type_code_3  
vehicle_type_code_4  
vehicle_type_code_5
```

3. Summarize the variables

```
missing_summary = (
    df.isna().mean().mul(100).rename("missing_percent").round(2).to_frame()
)
display(missing_summary)

numeric_summary = (
    df.select_dtypes(include="number")
    .describe()
    .T.loc[:, ["count", "min", "50%", "max"]]
    .rename(columns={"count": "n", "50%": "median"})
    .round(2)
)
display(numeric_summary)

for column in df.select_dtypes(include=["object", "string", "category"]):
    counts = df[column].value_counts(dropna=False)
    print(f"\n{column}: {len(counts):,} distinct values; 10 most frequent")
    print(counts.head(10).to_string())
```

	missing_percent
crash_date	0.00
crash_time	0.00
borough	28.46
zip_code	28.46
latitude	6.98
longitude	6.98
location	6.98
on_street_name	28.82
off_street_name	49.16
cross_street_name	71.18
number_of_persons_injured	0.00
number_of_persons_killed	0.00
number_of_pedestrians_injured	0.00
number_of_pedestrians_killed	0.00
number_of_cyclist_injured	0.00
number_of_cyclist_killed	0.00
number_of_motorist_injured	0.00
number_of_motorist_killed	0.00
contributing_factor_vehicle_1	0.48
contributing_factor_vehicle_2	23.39
contributing_factor_vehicle_3	90.87
contributing_factor_vehicle_4	97.26

	missing_percent
contributing_factor_vehicle_5	99.16
collision_id	0.00
vehicle_type_code_1	1.61
vehicle_type_code_2	33.47
vehicle_type_code_3	91.53
vehicle_type_code_4	97.43
vehicle_type_code_5	99.16

	n	min	median	max
latitude	1559.0	0.00	40.71	40.91
longitude	1559.0	-74.24	-73.92	0.00
number_of_persons_injured	1676.0	0.00	0.00	11.00
number_of_persons_killed	1676.0	0.00	0.00	4.00
number_of_pedestrians_injured	1676.0	0.00	0.00	7.00
number_of_pedestrians_killed	1676.0	0.00	0.00	4.00
number_of_cyclist_injured	1676.0	0.00	0.00	1.00
number_of_cyclist_killed	1676.0	0.00	0.00	0.00
number_of_motorist_injured	1676.0	0.00	0.00	11.00
number_of_motorist_killed	1676.0	0.00	0.00	1.00
collision_id	1676.0	4736561.00	4738146.50	4827469.00

crash_time: 737 distinct values; 10 most frequent

```
crash_time
0:00      34
15:00     19
19:00     18
13:00     17
14:30     15
10:00     14
23:00     13
22:30     13
15:30     13
16:00     13
```

borough: 6 distinct values; 10 most frequent

```
borough
NaN      477
BROOKLYN 416
QUEENS   336
MANHATTAN 213
BRONX    189
```

STATEN ISLAND 45

zip_code: 170 distinct values; 10 most frequent

zip_code	
<NA>	477
11207	26
11236	25
11208	24
11234	22
11101	21
11203	19
11212	19
11368	18
11226	17

location: 1,495 distinct values; 10 most frequent

location	
NaN	117
\n, \n(40.668507, -73.92561)	4
\n, \n(0.0, 0.0)	3
\n, \n(40.762856, -73.98942)	3
\n, \n(40.848038, -73.93285)	3
\n, \n(40.753326, -73.8718)	3
\n, \n(40.669476, -73.919975)	3
\n, \n(40.638268, -74.0788)	2
\n, \n(40.75702, -74.00494)	2
\n, \n(40.692383, -73.748726)	2

on_street_name: 653 distinct values; 10 most frequent

on_street_name	
NaN	483
BELT PARKWAY	28
FDR DRIVE	15
LONG ISLAND EXPRESSWAY	14
BROADWAY	13
BROOKLYN QUEENS EXPRESSWAY	12
3 AVENUE	10
GRAND CENTRAL PKWY	10
JAMAICA AVENUE	9
MAJOR DEEGAN EXPRESSWAY	8

off_street_name: 606 distinct values; 10 most frequent

off_street_name	
NaN	824
BROADWAY	16
BRUCKNER BOULEVARD	10

3 AVENUE	8
2 AVENUE	7
PARK AVENUE	6
NORTHERN BOULEVARD	6
SHORE PARKWAY	6
AVENUE OF THE AMERICAS	5
1 AVENUE	5

cross_street_name: 481 distinct values; 10 most frequent
cross_street_name

NaN	1193
1825 EASTCHESTER ROAD	2
2724 UNIVERSITY AVENUE	2
369 Hylan Boulevard	2
127-48 NORTHERN BOULEVARD	1
146 HERZL STREET	1
1700 FULTON STREET	1
4809 AVENUE J	1
225 VARICK STREET	1
1051 RIVERSIDE DRIVE	1

contributing_factor_vehicle_1: 41 distinct values; 10 most frequent
contributing_factor_vehicle_1

Unspecified	423
Driver Inattention/Distracted	404
Failure to Yield Right-of-Way	110
Following Too Closely	89
Unsafe Speed	74
Passing or Lane Usage Improper	67
Other Vehicular	60
Traffic Control Disregarded	51
Passing Too Closely	50
Alcohol Involvement	50

contributing_factor_vehicle_2: 23 distinct values; 10 most frequent
contributing_factor_vehicle_2

Unspecified	1088
NaN	392
Driver Inattention/Distracted	63
Other Vehicular	26
Unsafe Speed	19
Following Too Closely	13
Failure to Yield Right-of-Way	13
Passing or Lane Usage Improper	10
Traffic Control Disregarded	9
Pedestrian/Bicyclist/Other Pedestrian Error/Confusion	9

contributing_factor_vehicle_3: 5 distinct values; 10 most frequent

contributing_factor_vehicle_3

NaN	1523
Unspecified	147
Other Vehicular	3
Unsafe Speed	2
Aggressive Driving/Road Rage	1

contributing_factor_vehicle_4: 3 distinct values; 10 most frequent

contributing_factor_vehicle_4

NaN	1630
Unspecified	45
Aggressive Driving/Road Rage	1

contributing_factor_vehicle_5: 2 distinct values; 10 most frequent

contributing_factor_vehicle_5

NaN	1662
Unspecified	14

vehicle_type_code_1: 42 distinct values; 10 most frequent

vehicle_type_code_1

Sedan	769
Station Wagon/Sport Utility Vehicle	571
Taxi	49
Bike	37
Pick-up Truck	35
NaN	27
Motorcycle	24
Box Truck	23
Bus	22
E-Bike	14

vehicle_type_code_2: 40 distinct values; 10 most frequent

vehicle_type_code_2

NaN	561
Sedan	447
Station Wagon/Sport Utility Vehicle	320
Bike	76
Box Truck	34
Moped	32
Pick-up Truck	27
E-Scooter	24
Taxi	23
E-Bike	22

vehicle_type_code_3: 11 distinct values; 10 most frequent

vehicle_type_code_3	
NaN	1534
Sedan	73
Station Wagon/Sport Utility Vehicle	51
Pick-up Truck	5
Bus	5
Taxi	2
Tractor Truck Diesel	2
Moped	1
Bike	1
Van	1

vehicle_type_code_4: 6 distinct values; 10 most frequent

vehicle_type_code_4	
NaN	1633
Station Wagon/Sport Utility Vehicle	21
Sedan	19
Convertible	1
Motorcycle	1
Pick-up Truck	1

vehicle_type_code_5: 5 distinct values; 10 most frequent

vehicle_type_code_5	
NaN	1662
Sedan	9
Station Wagon/Sport Utility Vehicle	3
Box Truck	1
Bike	1

4. Validate longitude and latitude

Coordinates are invalid for this analysis when either value is missing or falls outside a conservative New York City bounding box. This correctly catches (0, 0), which is globally valid but not in New York City.

```
valid_geocode = (
    df["latitude"].between(40.45, 40.95)
    & df["longitude"].between(-74.30, -73.65)
)
invalid_geocode_count = int((~valid_geocode).sum())

df.loc[~valid_geocode, ["latitude", "longitude", "location"]] = np.nan
print(f"Invalid or missing coordinate pairs replaced with NA: {invalid_geocode_count}")
```

Invalid or missing coordinate pairs replaced with NA: 120

5. Validate NYC ZIP codes

The valid-code list comes from NYC DOHMH's Modified ZIP Code Tabulation Areas dataset. It is safer than accepting every integer in a broad range.

```
zip_reference = pd.read_csv(DATA_DIR / "nyc_modified_zip_codes.csv", dtype="string")
valid_zip_codes = {
    code.strip()
    for values in zip_reference["label"].dropna()
    for code in values.split(",")
}

df["zip_code"] = df["zip_code"].str.extract(r"(\d{5})", expand=False)
invalid_zip = df["zip_code"].notna() & ~df["zip_code"].isin(valid_zip_codes)
df.loc[invalid_zip, "zip_code"] = pd.NA

print(f"Non-NYC ZIP codes replaced with NA: {int(invalid_zip.sum())}")
```

Non-NYC ZIP codes replaced with NA: 2

6. Compare missing ZIP code and borough values

```
missing_zip = df["zip_code"].isna()
missing_borough = df["borough"].isna()

missing_comparison = pd.crosstab(
    missing_zip,
    missing_borough,
    rownames=["ZIP code missing"],
    colnames=["Borough missing"],
)
display(missing_comparison)
print(f"Missingness always co-occurs: {bool((missing_zip == missing_borough).all())}")
```

	Borough missing	
ZIP code missing	False	True
False	1197	0
True	2	477

Missingness always co-occurs: False

7. Fill ZIP code and borough from valid coordinates

The included NYC boundary files make the reverse lookup reproducible and avoid copying latitude into ZIP code or longitude into borough.

```
def compile_boundaries(geojson_path, property_name):
    with open(geojson_path, encoding="utf-8") as handle:
        features = json.load(handle)["features"]

    compiled = []
    for feature in features:
        geometry = feature["geometry"]
        if geometry is None:
            continue
        polygons = (
            [geometry["coordinates"]]
            if geometry["type"] == "Polygon"
            else geometry["coordinates"]
        )
        for rings in polygons:
            exterior = PolygonPath(np.asarray(rings[0]))
            holes = [PolygonPath(np.asarray(ring)) for ring in rings[1:]]
            compiled.append((feature["properties"][property_name], exterior, holes))
    return compiled

def lookup_boundary(longitude, latitude, boundaries):
    point = (longitude, latitude)
    for value, exterior, holes in boundaries:
        if exterior.contains_point(point) and not any(
            hole.contains_point(point) for hole in holes
        ):
            return value
    return None

borough_boundaries = compile_boundaries(
    DATA_DIR / "nyc_borough_boundaries.geojson", "boroname"
)
zip_boundaries = compile_boundaries(
    DATA_DIR / "nyc_modified_zip_boundaries.geojson", "modzcta"
)

fillable = (
    (df["zip_code"].isna() | df["borough"].isna())
```

```

    & df["latitude"].notna()
    & df["longitude"].notna()
)

for index, row in df.loc[fillable].iterrows():
    if pd.isna(row["zip_code"]):
        df.at[index, "zip_code"] = lookup_boundary(
            row["longitude"], row["latitude"], zip_boundaries
        )
    if pd.isna(row["borough"]):
        borough = lookup_boundary(
            row["longitude"], row["latitude"], borough_boundaries
        )
        df.at[index, "borough"] = borough.upper() if borough else pd.NA

print(f"Rows eligible for geographic filling: {int(fillable.sum())}")
print(f"ZIP codes still missing: {int(df['zip_code'].isna().sum())}")
print(f"Boroughs still missing: {int(df['borough'].isna().sum())}")

```

```

Rows eligible for geographic filling: 372
ZIP codes still missing: 107
Boroughs still missing: 107

```

8. Is location redundant?

`location` is a serialized pair derived from latitude and longitude. For analysis it is redundant, although an API may retain it for convenience and compatibility. Latitude and longitude should remain the authoritative numeric fields.

9. Crash frequency by hour

```

crash_time = pd.to_datetime(df["crash_time"], format="%H:%M", errors="coerce")
df["crash_hour"] = crash_time.dt.hour
hourly_frequency = df["crash_hour"].value_counts().sort_index()
display(hourly_frequency.rename("crashes").to_frame())

exact_midnight = int(df["crash_time"].eq("0:00").sum())
print(f"Crashes recorded at exactly 00:00: {exact_midnight}")

```

	crashes
crash_hour	
0	104
1	55

crashes	
crash_hour	
2	57
3	36
4	40
5	37
6	37
7	48
8	65
9	52
10	59
11	61
12	72
13	94
14	97
15	92
16	101
17	99
18	86
19	96
20	68
21	81
22	78
23	61

Crashes recorded at exactly 00:00: 34

An exact-midnight spike should not automatically be interpreted as bad luck or increased risk. 00:00 can also be used when the precise time is unknown or rounded, so the pattern warrants a data-quality question before a causal claim.

10. Reconcile injury and fatality totals

```
injury_components = [
    "number_of_pedestrians_injured",
    "number_of_cyclist_injured",
    "number_of_motorist_injured",
]
fatality_components = [
    "number_of_pedestrians_killed",
    "number_of_cyclist_killed",
    "number_of_motorist_killed",
]
```

```

df["calculated_injured_total"] = df[injury_components].sum(axis=1)
df["calculated_killed_total"] = df[fatality_components].sum(axis=1)

injury_mismatches = int(
    df["number_of_persons_injured"].ne(df["calculated_injured_total"]).sum()
)
fatality_mismatches = int(
    df["number_of_persons_killed"].ne(df["calculated_killed_total"]).sum()
)
print(f"Injury-total mismatches: {injury_mismatches}")
print(f"Fatality-total mismatches: {fatality_mismatches}")

```

```

Injury-total mismatches: 43
Fatality-total mismatches: 0

```

The totals should not be removed merely because they are derivable: keeping them supports auditing, but the provider should document and resolve any discrepancies.

11. Standardize the primary contributing factor

```

initial_factors = df["contributing_factor_vehicle_1"].value_counts(dropna=False)
display(initial_factors.rename("count").to_frame())

df["contributing_factor_vehicle_1"] = (
    df["contributing_factor_vehicle_1"].str.strip().str.upper()
)
standardized_factors = df["contributing_factor_vehicle_1"].value_counts(dropna=False)
display(standardized_factors.rename("count").to_frame())

```

contributing_factor_vehicle_1	count
Unspecified	423
Driver Inattention/Distracted	404
Failure to Yield Right-of-Way	110
Following Too Closely	89
Unsafe Speed	74
Passing or Lane Usage Improper	67
Other Vehicular	60
Traffic Control Disregarded	51
Passing Too Closely	50
Alcohol Involvement	50
Driver Inexperience	45
Backing Unsafely	45

contributing_factor_vehicle_1	count
Turning Improperly	40
Unsafe Lane Changing	22
Pedestrian/Bicyclist/Other Pedestrian Error/Confusion	22
Reaction to Uninvolved Vehicle	14
Aggressive Driving/Road Rage	13
View Obstructed/Limited	13
Pavement Slippery	9
Oversized Vehicle	8
NaN	8
Fell Asleep	8
Brakes Defective	6
Lost Consciousness	5
Tire Failure/Inadequate	5
Outside Car Distraction	4
Glare	3
Failure to Keep Right	3
Illnes	3
Steering Failure	2
Pavement Defective	2
Fatigued/Drowsy	2
Tow Hitch Defective	2
Accelerator Defective	2
Tinted Windows	2
Cell Phone (hand-Held)	2
Passenger Distraction	2
Driverless/Runaway Vehicle	2
Obstruction/Debris	2
Lane Marking Improper/Inadequate	1
Cell Phone (hands-free)	1

contributing_factor_vehicle_1	count
UNSPECIFIED	423
DRIVER INATTENTION/DISTRACTION	404
FAILURE TO YIELD RIGHT-OF-WAY	110
FOLLOWING TOO CLOSELY	89
UNSAFE SPEED	74
PASSING OR LANE USAGE IMPROPER	67
OTHER VEHICULAR	60
TRAFFIC CONTROL DISREGARDED	51
PASSING TOO CLOSELY	50

contributing_factor_vehicle_1	count
ALCOHOL INVOLVEMENT	50
DRIVER INEXPERIENCE	45
BACKING UNSAFELY	45
TURNING IMPROPERLY	40
UNSAFE LANE CHANGING	22
PEDESTRIAN/BICYCLIST/OTHER PEDESTRIAN ERROR/CONFUSION	22
REACTION TO UNINVOLVED VEHICLE	14
AGGRESSIVE DRIVING/ROAD RAGE	13
VIEW OBSTRUCTED/LIMITED	13
PAVEMENT SLIPPERY	9
OVERSIZED VEHICLE	8
NaN	8
FELL ASLEEP	8
BRAKES DEFECTIVE	6
LOST CONSCIOUSNESS	5
TIRE FAILURE/INADEQUATE	5
OUTSIDE CAR DISTRACTION	4
GLARE	3
FAILURE TO KEEP RIGHT	3
ILLNES	3
STEERING FAILURE	2
PAVEMENT DEFECTIVE	2
FATIGUED/DROWSY	2
TOW HITCH DEFECTIVE	2
ACCELERATOR DEFECTIVE	2
TINTED WINDOWS	2
CELL PHONE (HAND-HELD)	2
PASSENGER DISTRACTION	2
DRIVERLESS/RUNAWAY VEHICLE	2
OBSTRUCTION/DEBRIS	2
LANE MARKING IMPROPER/INADEQUATE	1
CELL PHONE (HANDS-FREE)	1

12. Recommendations to the data provider

The provider should validate NYC coordinate bounds, publish a code list for ZIP and categorical fields, distinguish unknown times from true midnight, and document how injury totals are reconciled with pedestrian, cyclist, and motorist subtotals. Stable validation rules would make revisions easier to audit.